

RaspberryPi3EGLFS Cross-Compile and Qt Setup Guide

Azar 1404

RaspberryPi3EGLFS Cross-Compile and Qt Setup Guide

1. Prepare Host PC

- Create working directory and get toolchain:

```
mkdir ~/raspi
cd ~/raspi
git clone https://github.com/raspberrypi/tools
```

- We used Linaro toolchain:

<https://releases.linaro.org/components/toolchain/binaries/>

2. Create and Sync Sysroot

```
mkdir /path/to/rpi-sysroot /path/to/rpi-sysroot/usr /path/to/rpi-sysroot/opt
rsync -avz root@rpi3device:/lib /path/to/rpi-sysroot
rsync -avz root@rpi3device:/usr/include /path/to/rpi-sysroot/usr
rsync -avz root@rpi3device:/usr/lib /path/to/rpi-sysroot/usr
rsync -avz root@rpi3device:/opt/vc /path/to/rpi-sysroot/opt
rsync -avz root@rpi3device:/usr/local/qt5pi
/path/to/rpi-sysroot/usr/local/qt5pi
```

3. Fix Sysroot Symlinks

```
wget https://raw.githubusercontent.com/Kukkimonsuta/rpi-
buildqt/master/scripts/utils/sysroot-relativelinks.py
chmod +x sysroot-relativelinks.py
./sysroot-relativelinks.py /path/to/rpi-sysroot
```

4. Get Qt and Configure

Qt Shell Commands

Clone Qt Base

```
git clone git://code.qt.io/qt/qtbase.git -b <qt-version>
cd qtbase
```

or use qt-everywhere-src-.tar.xz and its qtbase subdirectory.

Configure Qt for Cross-Compilation

```
./configure -release -opengl es2 -device <rpi-version> \  
-device-option CROSS_COMPILE=/absolute/path/to/linaro/bin/arm-linux-gnueabihf-\  
\  
-sysroot ~/raspi/sysroot -opensource -confirm-license -make libs \  
-prefix /usr/local/qt5pi -extprefix ~/raspi/qt5pi -hostprefix ~/raspi/qt5 -v
```

Configure Qt for Cross-Compilation (just tools)

```
./configure -release -opengl es2 -device <rpi-version> \  
-device-option CROSS_COMPILE=/absolute/path/to/linaro/bin/arm-linux-gnueabihf-\  
\  
-sysroot ~/raspi/sysroot -opensource -confirm-license -make tools \  
-prefix /usr/local/qt5pi -extprefix ~/raspi/qt5pi -hostprefix ~/raspi/qt5 -v
```

Build and Install Qt

```
make  
make install
```

- Clean build if needed:

```
git clean -dfx
```

What I Used

qt-everywhere-src-5.12.11.tar.xz

downloaded and extracted.

Configure Qt for Cross-Compilation

```
./configure -release -opengl es2 -device linux-rasp-pi3-g++ -device-option  
CROSS_COMPILE=/absolute/path/to/bin/arm-linux-gnueabihf- -sysroot  
/absolute/path/to/rpi-sysroot/ -opensource -confirm-license -make libs -skip  
qtscript -skip qtwayland -skip qtdatavis3d -skip qtwebengine -nomake examples  
-nomake tests -prefix /usr/local/qt5pi -extprefix /absolute/path/to/ext-qt5pi  
-hostprefix /absolute/path/to/host-qt5 -pkg-config -no-use-gold-linker -v
```

- **Note:** no relative path for any paths just use absolute paths.
- **Note:** add `#include <limits>` if you get errors about `std::numeric_limits` during configure:
 - qt-everywhere-src-5.12.12/qtbase/src/corelib/global/qendian.h
 - qt-everywhere-src-5.12.12/qtbase/src/corelib/tools/qbytearraymatche
 - qt-everywhere-src-5.12.12/qtbase/src/corelib/kernel/qvariant.h
 - qt-everywhere-src-5.12.12/qtdeclarative/src/qml/jsruntime/qv4proper
 - qt-everywhere-src-5.12.12/qttools/src/qdoc/generator.cpp

Configure Qt for Cross-Compilation (just tools)

```
./configure -release -opengl es2 -device linux-rasp-pi3-g++ -device-option  
CROSS_COMPILE=/home/danial/Code/novin-med/projects/1_rpi_qt/rpi3/gcc-linaro-  
7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin/arm-linux-gnueabihf- -sysroot  
/home/danial/Code/novin-med/projects/1_rpi_qt/rpi3/rpi-sysroot -opensource  
-confirm-license -make tools -skip qtscript -skip qtwayland -skip qtdatavis3d  
-skip qtwebengine -nomake examples -nomake tests -prefix /usr/local/qt5pi  
-hostprefix /home/danial/Code/novin-med/projects/1_rpi_qt/rpi3/qt5 -pkg-config  
-no-use-gold-linker
```

Build and Install Qt

```
make  
make install
```

5. Build and Test Example

u can use device source code for this step.

```
cd qtbase/examples/opengl/qopenglwidget  
/path/to/qmake  
make  
scp qopenglwidget root@rpi3device:/home/pi
```

Additional Notes

- For opengl shader error (black screen when deploy app), add
QMAKE_LFLAGS += -Wl,-rpath,/opt/vc/lib -Wl,-rpath,/usr/local/qt5pi/lib
to .pro file for setting vc libraries path above arm-gnueabihf libraries path
(EGLFS ES2)
- For low resolution issues, add `disable_overscan=1` to `/boot/config.txt`
and use `tvservice/fbset` to set resolution.
- Enable Qt logging for debugging:

```
export QT_LOGGING_RULES=qt.qpa.*=true  
./qopenglwidget
```
- Check `config.summary` for enabled features (Evdev, FontConfig, FreeType,
libinput, EGLFS, EGLFS Raspberry Pi, LinuxFB, udev, xkbcommon-evdev).

Troubleshooting

- If EGLFS is missing, use `linux-rasp-pi-g++ mkspec` and copy Pi 3 build
flags.
- Fix VC paths in `qmake.conf` if needed.

Environment Variables

If Qt apps fail to find platform plugins:

```
export QT_QPA_PLATFORM=eglfs
export QT_QPA_PLATFORM_PLUGIN_PATH=/usr/local/qt5pi/plugins/platforms
export LD_LIBRARY_PATH=/usr/local/qt5pi/lib
```

Qt Creator Setup

- Add device, compiler, debugger, Qt version, and kit in Qt Creator.
- Use deploy steps and custom run configuration if needed.

Sense HAT

- Use unofficial Qt Sense HAT module for sensors and LEDs.

Qt Multimedia

- GStreamer-based multimedia may be unstable.
- Try OpenMAX for video/camera.
- Install GStreamer dependencies before building Qt Multimedia:

```
sudo apt-get install gstreamer1.0-omx libgstreamer1.0-dev
libgstreamer-plugins-base1.0-dev
```
- Sync headers/libs to sysroot and re-run symlink script.

Example: Build Qt Multimedia with GStreamer 1.0

```
~/raspi/qt5/bin/qmake -r GST_VERSION=1.0
make
make install
```

- For OpenMAX debugging:

```
export GST_DEBUG=omx:4
```